

AI Seller Management Platform

Updated Complete Project Documentation

React + Node.js + Express + MongoDB + Firebase Authentication

Amazon • Flipkart • Myntra • Own Website • Offline POS • AI Product Management • Inventory •
Orders • Trends

Version: September 2026

1. Project Overview

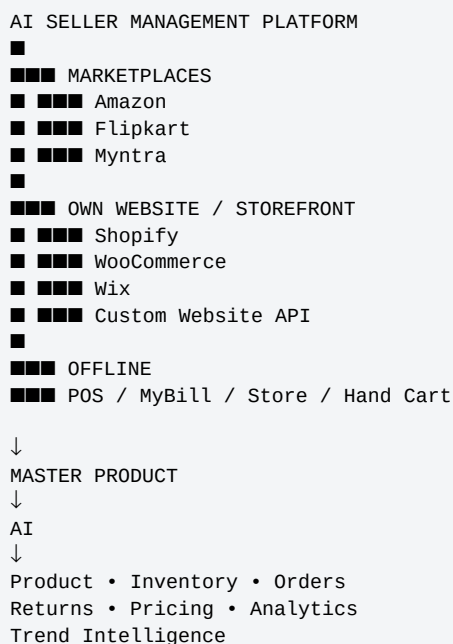
The AI Seller Management Platform is a centralized commerce management system for sellers who operate across marketplaces, their own ecommerce website, and offline stores. The seller creates or imports a product once; AI assists with product content and channel-specific information, while the platform manages publishing, inventory, orders, returns, pricing, analytics, and trend intelligence from one dashboard.

The architecture uses a **Master Product** as the single source of product truth. Marketplace and website integrations act as channels connected to that master catalog.

2. Main Objectives

- Connect Amazon, Flipkart, and Myntra accounts through supported official authorization/integration mechanisms.
- Connect a seller's own ecommerce website through supported platform APIs or a custom integration API.
- Upload product images/details once and use AI to generate titles, descriptions, categories, attributes, keywords and SEO content.
- Publish the approved master product to multiple sales channels.
- Maintain centralized inventory across marketplaces, own website and offline POS.
- Synchronize orders, returns, prices and stock wherever the connected channel APIs permit.
- Provide fashion and commerce trend intelligence such as rising categories, colors and styles.
- Keep credentials, tokens, permissions and audit events secure.

3. Channel Architecture



The important architectural addition is the **Own Website / Storefront Integration** layer. A seller does not need to maintain product data separately for every channel. The platform maps the master product into each connected channel's data model.

4. Own Website Integration

A seller may already have an ecommerce website such as Shopify, WooCommerce, Wix, or a custom-built store. The platform should treat the website as another sales channel rather than as a separate product database.

Connection flow

```

Seller Dashboard
↓
Connect Own Website
↓
Choose Platform
■■■ Shopify → Official authorization / OAuth
■■■ WooCommerce → Store URL + API credentials
■■■ Wix → Supported app/API authorization
■■■ Custom → Seller's integration API / credentials
↓
Validate Connection
↓
Store encrypted connection data
↓
Initial Product + Inventory Sync
↓
Connected Website Channel

```

Important: saving only a public website URL is not enough to create products automatically. The website must expose an official API/app integration, or the seller's custom backend must expose an integration endpoint/webhook contract.

For Shopify, current official documentation describes the GraphQL Admin API as the current product-management path for new public apps; product creation requires appropriate product-write permissions.

■cite■turn0search1■turn0search3■

Shopify also documents product synchronization use cases and supports product data including variants, media, SEO metadata, categories and tags. ■cite■turn0search2■

5. AI Product Creation Pipeline

```

Seller uploads:
Images + Product Name + Basic Details + Price + SKU
↓
AI Processing
↓
████████████████████████████████████████████████████████████████████████████████
██  ██  ███
Image AI Text AI Attribute AI
██  ██  ███
████████████████████████████████████████████████████████████████████████████████
↓
Master Product
↓
Seller Review
↓
Channel-specific Mapping
↓
Amazon / Flipkart / Myntra / Website / POS

```

AI-generated content should remain reviewable. The system should not blindly publish uncertain AI-generated attributes. Seller approval can be recorded in the audit trail.

6. Master Product Model

```
{
  sellerId,
  sku,
  title,
  description,
  category,
  brand,
  images: [],
  variants: [
    { sku, size, color, price, stock }
  ],
  attributes: {},
  seo: {
    title,
    description,
```

```

keywords
},
channelMappings: [],
createdAt,
updatedAt
}

```

The master product is channel-neutral. Each connector transforms it into the schema expected by its destination channel.

7. Website Product Publishing

```

Master Product
↓
Channel Mapper
↓
Website Adapter
↓
Official Website API
↓
Create / Update Product
↓
Store External Product ID
↓
Sync Status = PUBLISHED

```

Example custom website integration contract:

```

POST /api/integration/products

{
  "sku": "BC001",
  "name": "Handcrafted Bamboo Basket",
  "description": "...",
  "price": 899,
  "stock": 25,
  "images": [],
  "category": "Bamboo Products"
}

```

The platform should store the external product identifier so future updates modify the existing product instead of creating duplicates.

8. Unified Inventory

```

MASTER STOCK = 100

Amazon -10
Flipkart -5
Myntra -0
Own Website -8
Offline -7
-----
Current Stock = 70

```

Inventory changes should be recorded as ledger transactions. Channel-specific stock values can be synchronized from the master inventory according to the channel's API rules.

```

{
  storeId,
  productId,
  variantId,
  type: "SALE | PURCHASE | RETURN | ADJUSTMENT | SYNC",
  channel: "AMAZON | FLIPKART | MYNTRA | OWN_WEBSITE | OFFLINE",
  quantityChange,
  referenceId,
  previousStock,
  newStock,
  createdAt
}

```

9. Website Order Synchronization

```
Own Website Order
↓
Webhook / Scheduled API Sync
↓
Integration Adapter
↓
Order Normalization
↓
Master Order in MongoDB
↓
Inventory Transaction
↓
Updated Stock
↓
Other Connected Channels
```

Where supported, webhooks should be preferred for near-real-time order events. Scheduled reconciliation can be used as a fallback to detect missed events.

10. Marketplace + Website Adapter Architecture

```
backend/
  ■■■ integrations/
  ■ ■■■ marketplaces/
  ■ ■ ■■■ amazon/
  ■ ■ ■■■ flipkart/
  ■ ■ ■■■ myntra/
  ■ ■■■ ecommerce/
  ■ ■ ■■■ shopify/
  ■ ■ ■■■ woocommerce/
  ■ ■ ■■■ wix/
  ■ ■ ■■■ custom/
  ■ ■■■ pos/
  ■ ■■■ mybill/
  ■
  ■■■ products/
  ■■■ inventory/
  ■■■ orders/
  ■■■ returns/
  ■■■ ai/
  ■■■ trends/
  ■■■ analytics/
  ■■■ notifications/
```

Recommended common adapter methods:

```
connect()
disconnect()
getProducts()
publishProduct()
updateProduct()
updateInventory()
updatePrice()
fetchOrders()
fetchReturns()
registerWebhooks()
sync()
```

11. Connected Channel Data Model

```
{
  sellerId,
  type: "MARKETPLACE | OWN_WEBSITE | POS",
  provider: "AMAZON | FLIPKART | MYNTRA | SHOPIFY | WOOCOMMERCE | WIX | CUSTOM",
  status: "CONNECTED",
  credentials: "ENCRYPTED",
  lastSyncAt,
  metadata: {}
}
```

Sensitive access tokens and credentials should be encrypted and should never be exposed to the React client.

12. Suggested UI — Marketplace & Website Connections

Marketplaces & Channels

Amazon [Connected]

Flipkart [Connect]

Myntra [Connect]

Own Website

[+ Connect Website]

Offline POS

[+ Connect POS]

Website connection wizard

Connect Your Website

What platform powers your website?

☒ Shopify

☒ WooCommerce

☒ Wix

☒ Custom Website

☒ Other

[Continue]

Connected website card

Bhartiye Crafts

<https://bhartiynecrafts.com>

Products: 248

Orders: 32

Stock: Synced ✓

Last Sync: 2 minutes ago

[Sync Now] [Manage] [Disconnect]

The Bhartiye Crafts public storefront can be used as a conceptual example of an existing ecommerce website, but the actual platform behind a seller's website must be confirmed from owner/admin access or technical documentation before selecting the connector.

13. Frontend Structure

```
frontend/  
  ■■■ src/  
    ■ ■■■ app/  
    ■ ■■■ components/  
    ■ ■■■ layouts/  
    ■ ■■■ pages/  
    ■ ■ ■■■ auth/  
    ■ ■ ■■■ dashboard/  
    ■ ■ ■■■ products/  
    ■ ■ ■■■ ai/  
    ■ ■ ■■■ marketplaces/  
    ■ ■ ■■■ websites/  
    ■ ■ ■■■ orders/  
    ■ ■ ■■■ inventory/  
    ■ ■ ■■■ pos/  
    ■ ■ ■■■ trends/  
    ■ ■ ■■■ analytics/  
    ■ ■ ■■■ settings/  
    ■ ■■■ services/  
    ■ ■■■ store/  
    ■ ■■■ hooks/  
    ■ ■■■ utils/  
    ■ ■■■ config/  
    ■■■ .env
```

14. Backend Structure

```
backend/  
  ■■■■ src/  
  ■ ■■■■ config/  
  ■ ■■■■ middleware/  
  ■ ■■■■ modules/  
  ■ ■ ■■■■ users/  
  ■ ■ ■■■■ stores/  
  ■ ■ ■■■■ products/  
  ■ ■ ■■■■ inventory/  
  ■ ■ ■■■■ orders/  
  ■ ■ ■■■■ offlineSales/  
  ■ ■ ■■■■ returns/  
  ■ ■ ■■■■ ai/  
  ■ ■ ■■■■ trends/  
  ■ ■ ■■■■ analytics/  
  ■ ■ ■■■■ notifications/  
  ■ ■■■■ integrations/  
  ■ ■ ■■■■ marketplaces/  
  ■ ■ ■■■■ ecommerce/  
  ■ ■ ■■■■ pos/  
  ■ ■■■■ jobs/  
  ■ ■■■■ services/  
  ■ ■■■■ models/  
  ■ ■■■■ validators/  
  ■ ■■■■ routes/  
  ■■■■ server.js
```

15. Technology Stack

- Frontend: React + Vite + Tailwind CSS
- Backend: Node.js + Express.js
- Database: MongoDB / MongoDB Atlas
- Authentication: Firebase Authentication
- AI: image + text + attribute generation services
- Payments/subscriptions: Razorpay if monetization is required
- Realtime/events: Webhooks and optional Socket.IO for application events
- Hosting: Vercel for frontend and Render/Railway or equivalent for backend

16. Firebase Authentication

```
React  
↓  
Firebase Authentication  
↓  
Firebase ID Token  
↓  
Authorization Header  
↓  
Express authMiddleware  
↓  
Verify Firebase ID Token  
↓  
Resolve Seller / Store  
↓  
Protected API
```

Firebase authenticates users of the platform. Marketplace and website access tokens are separate credentials managed by the integration layer.

17. MongoDB Collections

Recommended collections:

users, stores, marketplaceAccounts, connectedChannels, products, productVariants, categories, marketplaceListings, websiteListings, inventory, inventoryTransactions, orders, orderItems, offlineSales, customers, suppliers, purchases, returns, aiJobs, trendSignals, notifications, subscriptions, auditLogs

18. Suggested API Structure

```
AUTH
POST /api/auth/profile
GET /api/auth/me

CHANNELS
GET /api/channels
POST /api/channels/:provider/connect
GET /api/channels/:provider/callback
POST /api/channels/:provider/disconnect
POST /api/channels/:provider/sync

WEBSITES
GET /api/websites
POST /api/websites/connect
POST /api/websites/:id/sync
GET /api/websites/:id/status

PRODUCTS
POST /api/products
GET /api/products
GET /api/products/:id
PATCH /api/products/:id
DELETE /api/products/:id

AI
POST /api/ai/product/generate
POST /api/ai/product/attributes
POST /api/ai/image/enhance

PUBLISH
POST /api/products/:id/publish
POST /api/products/:id/publish/:channelId

ORDERS
GET /api/orders
POST /api/orders/sync

INVENTORY
GET /api/inventory
POST /api/inventory/adjust
POST /api/inventory/sync

TRENDS
GET /api/trends
GET /api/trends/categories
GET /api/trends/colors
```

19. Security Requirements

- Use HTTPS for every integration and API request.
- Verify Firebase ID tokens on protected backend routes.
- Never ask sellers to enter or store marketplace OTP/passwords in your application.
- Use official OAuth/authorization/API mechanisms wherever the provider supports them.
- Encrypt sensitive marketplace and website credentials/tokens at rest.
- Keep secrets in environment variables or a secrets manager.
- Use rate limiting, schema validation, RBAC, centralized error handling and audit logs.

- Use idempotency for publishing, inventory synchronization and order ingestion.
- Verify webhook signatures where supported.
- Keep channel-specific permissions as narrow as possible.

20. Fashion & Trend Intelligence

- Trending products and categories
- Rising / stable / declining trend signals
- Popular colors, styles and attributes
- Seller's own sales trend vs broader market signals
- AI-assisted product discovery
- Suggested product opportunities based on catalog gaps and observed trends

21. End-to-End Seller Flow

1. Seller signs in with Firebase
2. Creates store profile
3. Connects Amazon / Flipkart / Myntra
4. Connects own website if available
5. Uploads product image + basic details
6. AI generates product content
7. Seller reviews and approves
8. Master product is saved
9. Channel adapters map the product
10. Product is published to selected channels
11. Orders arrive from marketplaces / website / POS
12. Inventory ledger updates master stock
13. Stock/price changes are synchronized
14. Returns and analytics are consolidated
15. Trend engine provides market insights

22. MVP Scope

MVP should include:

- Firebase login and seller profile
- Master product CRUD
- AI title/description/attribute generation
- Amazon/Flipkart/Myntra connection framework
- Own Website connection framework with at least one concrete provider connector
- Website product publishing and sync
- Central inventory ledger
- Unified order dashboard
- Basic analytics
- Audit logs and secure credential handling

23. Development Roadmap

Phase 1 — Foundation: Firebase authentication, seller/store setup, dashboard shell and channel UI.

Phase 2 — Product Core: master product, variants, categories, media and inventory ledger.

Phase 3 — AI: image processing, title/description generation, attributes and review workflow.

Phase 4 — Marketplace Connectors: Amazon, Flipkart and Myntra according to approved APIs/permissions.

Phase 5 — Own Website: Shopify/WooCommerce/Wix/custom adapter, product publishing, order sync and webhooks.

Phase 6 — Unified Operations: inventory, price sync, returns, POS and notifications.

Phase 7 — Intelligence: trends, analytics, profit reporting and AI recommendations.

Phase 8 — Production Hardening: security audit, idempotency, retries, observability, QA and deployment.

24. Production Design Principles

- Master data first: one canonical product and inventory ledger.
- Adapters isolate provider-specific API differences.
- Event-driven synchronization where webhooks are available.
- Scheduled reconciliation for reliability.
- Every external operation should be retry-safe and idempotent.
- Every important automated action should be observable through logs/audit history.
- AI assists the seller; seller approval remains available before publication.
- Provider API capabilities and permissions must be verified before implementation.

25. Final Architecture Summary



Conclusion

The updated platform is designed as a true omnichannel seller system: marketplaces, the seller's own website, and offline sales all connect to one master product and inventory system. The new Own Website / Storefront Integration module allows AI-generated products to be mapped and published to supported ecommerce platforms through their official APIs or a controlled custom integration, while keeping orders and inventory synchronized back into the central system.